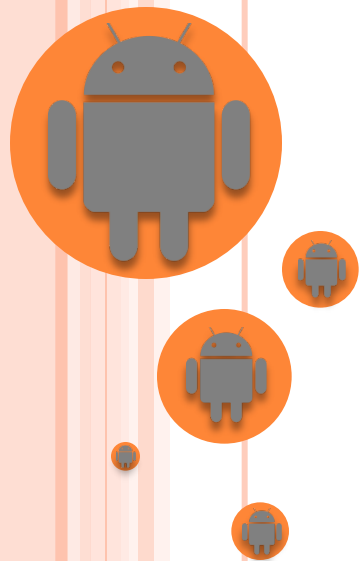
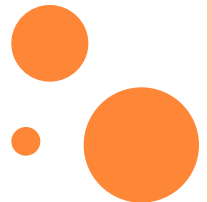


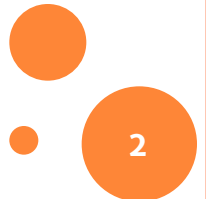


TRABAJO EN SEGUNDO PLANO



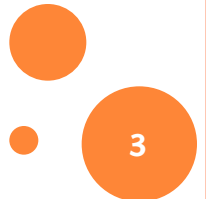
TRABAJO EN SEGUNDO PLANO

- WorkManager
 - DownloadManager
 - AlarmManager
- Complementarios a los servicios

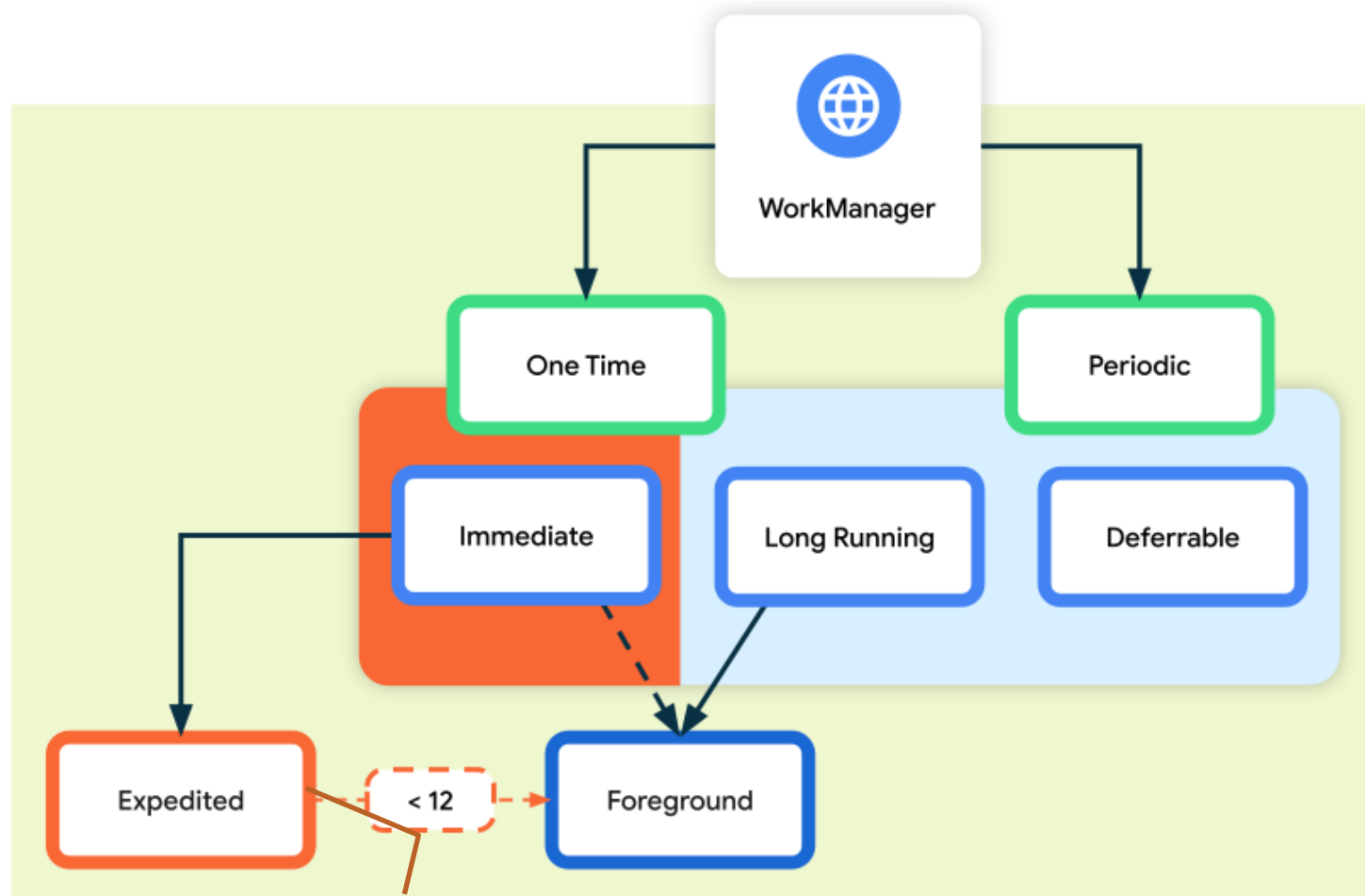


WORKMANAGER

- WorkManager es una API incorporada en el API 28 para gestionar tareas programables (***persistent work: se mantiene programado a pesar de reinicios de la app o del sistema***)
 - Para APIs menores, WorkManager usa por debajo:
 - JobScheduler (para API ≥ 23)
 - Custom AlarmManager + BroadcastReceiver (para lograr compatibilidad con API 14-22)
- Permite lanzar tareas:
 - De única ejecución
 - De ejecución repetitiva
 - Larga ejecución (más de 10 minutos)
 - Programable a posteriori (***Deferrable***)

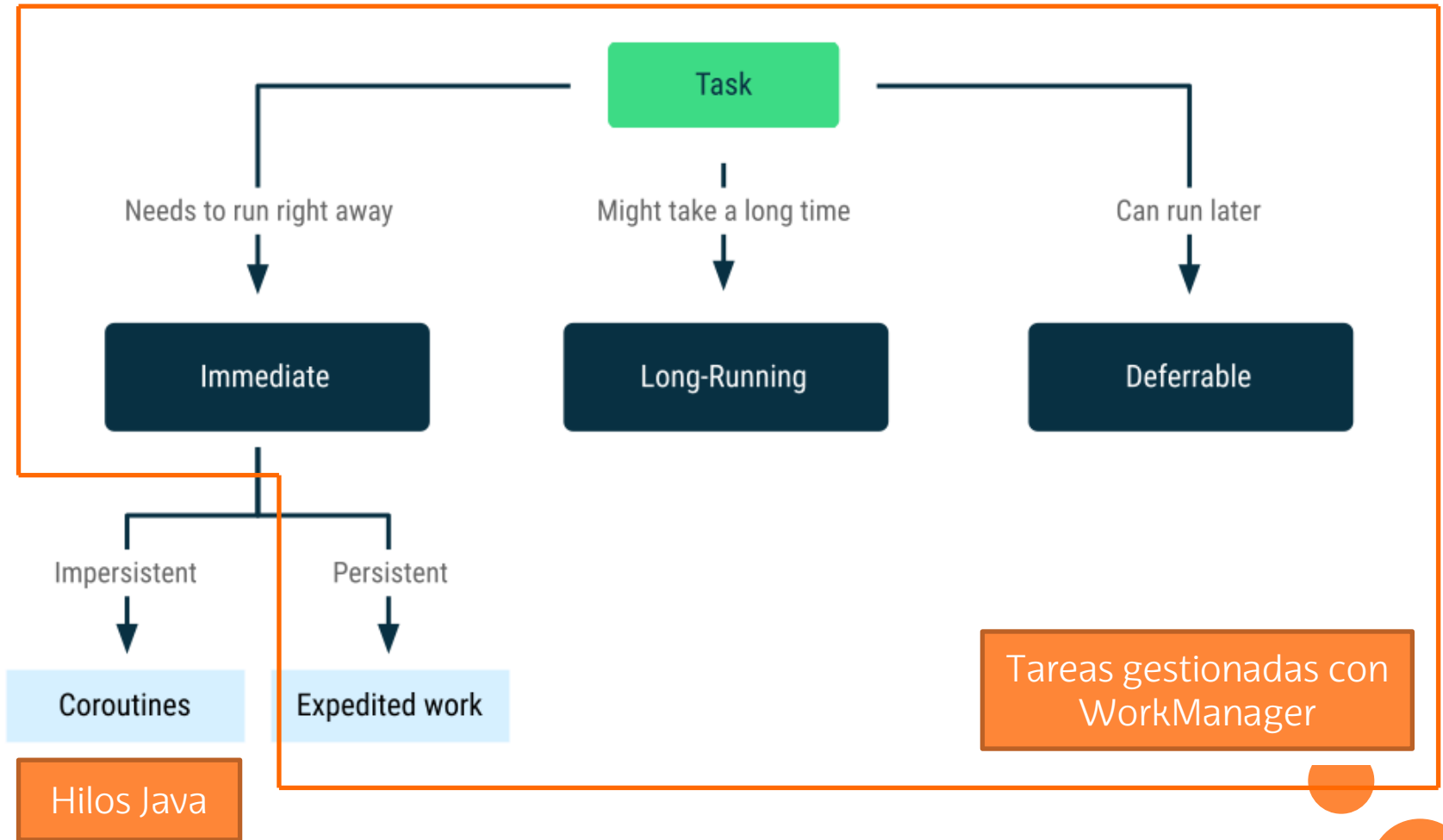


WORKMANAGER



Para trabajos acelerados, se llama a `setExpedited`, o `setForeground` en Android < 12

WORKMANAGER



WORKMANAGER

- Para usar WorkManager, hay que incluirla entre las dependencias del módulo en Gradle

```
dependencies {  
    ...  
    implementation 'androidx.work:work-runtime:2.11.1'  
    ...  
}
```

Se añade en el fichero build.gradle DEL
MÓDULO (app)



WORKMANAGER

- Para definir una tarea hay que crear una clase que extienda a **Worker**
 - Del paquete `androidx.work`
- Hay que sobrescribir el método **doWork()** que es donde se realizará la tarea de forma asíncrona
 - Puede devolver
 - **Result.success()** para indicar que la tarea se ha ejecutado correctamente
 - **Result.failure()** para indicar que algo ha fallado en la ejecución
 - **Result.retry()** para indicar que hay que reintentar la ejecución en otro momento

```
public class Tarea extends Worker {  
  
    public Result doWork() {  
        ...  
        return Result.success();  
    }  
}
```

Trabajo a realizar

Resultado a devolver

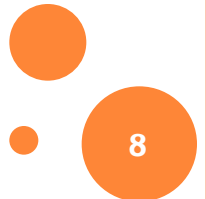


WORKMANAGER

- Un trabajo se puede ejecutar
 - Una vez → *OneTimeWorkRequest*
 - De manera periódica → *PeriodicWorkRequest*
- La ejecución de un trabajo puede ir acompañada de restricciones
 - Que el dispositivo tenga red
 - Que el dispositivo esté en modo carga
 - Que el nivel de batería no sea bajo
 - Que el dispositivo esté inactivo (idle)
 - Que el espacio de almacenamiento sea suficiente

Algunas dependen
del SDK objetivo

<https://developer.android.com/reference/androidx/work/Constraints.Builder>



WORKMANAGER

○ Ejemplo:

- Ejecutar un trabajo una única vez cuando el dispositivo esté cargando

```
Constraints restricciones = new Constraints.Builder()  
    .setRequiresCharging(true)  
    .build();
```

Restricciones a cumplir

```
OneTimeWorkRequest trabajoPuntual =  
    new OneTimeWorkRequest.Builder(Tarea.class)  
        .setConstraints(restricciones)  
        .build();
```

Se establecen las
restricciones

Clase de la tarea

```
WorkManager.getInstance(this).enqueue(trabajoPuntual);
```

Contexto

Encolar el trabajo

El momento de ejecución del trabajo lo decidirá el sistema operativo teniendo en cuenta las restricciones



WORKMANAGER

○ Ejemplo:

- Ejecutar un trabajo cada 25 minutos cuando haya cobertura

```
Constraints restricciones = new Constraints.Builder()
    .setRequiredNetworkType(NetworkType.CONNECTED)
    .build();

PeriodicWorkRequest trabajoRepetitivo =
    new PeriodicWorkRequest.Builder(Tarea.class, 25, TimeUnit.MINUTES)
        .setConstraints(restricciones)
        .build();

WorkManager.getInstance(this).enqueue(trabajoRepetitivo);
```

Restricciones a cumplir

Clase de la tarea

Cada cuanto se repite

Unidad

Se establecen las restricciones

Encolar el trabajo

El momento de ejecución del trabajo lo decidirá el sistema operativo teniendo en cuenta las restricciones y la periodicidad (Mínimo 15 minutos)



WORKMANAGER

- Se puede enviar información a la tarea a través de objetos *Data*

```
Data datos = new Data.Builder()
    .putString("Clave", "valor")
    .putInt("Clave2", 5)
    .build();
```

Pares clave, valor

```
PeriodicWorkRequest trabajoRepetitivo =
    new PeriodicWorkRequest.Builder(Tarea.class, 15, TimeUnit.MINUTES)
        .setConstraints(restricciones)
        .setInputData(datos)
        .build();
```

Establecer los datos

```
public Result doWork() {
    String valor= getInputData().getString("Clave");
    Integer num= getInputData().getInt("Clave2", 0);
    ...
}
```

Recuperar los datos

Valor por defecto



WORKMANAGER

- Se puede recuperar información de la tarea a través de objetos *Data*:

```
public Result doWork() {  
    ...  
    Data resultados = new Data.Builder()  
        .putInt("resultado", 42)  
        .build();  
    return Result.success(resultados);  
}
```

Pares clave, valor

Devolver los resultados

- En la actividad desde donde se ha encolado la tarea:

```
WorkManager.getInstance(this).getWorkInfoByIdLiveData(trabajoPuntual.getId())  
    .observe(this, status -> {  
        if (status != null && status.getState().isFinished()) {  
            Integer result = status.getOutputData().getInt("resultado", 0);  
            ...  
        }  
    });
```

El trabajo

Recuperar el resultado



WORKMANAGER

- Si la ejecución no es la deseada se puede indicar que hay que volver a repetir la ejecución mediante **Result.retry()**
- Se intentará ejecutar de nuevo la tarea en base a una **Backoff Policy**:
 - Lo intenta pasado un tiempo de modo lineal
 - Incrementa el tiempo de modo exponencial
- Establecer una **Backoff Policy**

```
OneTimeWorkRequest trabajoPuntual =  
    new OneTimeWorkRequest.Builder(Tarea.class)  
        .setBackoffCriteria(BackoffPolicy.LINEAR, 3, TimeUnit.SECONDS)  
        .build();
```

Incrementar el tiempo entre intentos de manera lineal y de 3 en 3 segundos



WORKMANAGER

- Se puede cancelar un trabajo concreto o todos los establecidos

```
WorkManager.getInstance().cancelWorkById(trabajoPuntual.getId());
```

Cancelar un trabajo

Cancelar todos los trabajos

```
WorkManager.getInstance().cancelAllWork();
```



WORKMANAGER

○ Ejercicio 1

- Crear una aplicación que establezca un trabajo que se ejecute 10 segundos después de abrir la aplicación .
 - El trabajo lo único que hará será lanzar una notificación
 - Para hacer que un trabajo se ejecute pasado 10 segundos:

```
.setInitialDelay(10, TimeUnit.SECONDS)
```

En el builder del trabajo

- Crear otro trabajo que se ejecute cada 20 segundos si el dispositivo está cargando
 - El trabajo abrirá una actividad de la aplicación (ver siguiente diapositiva)



WORKMANAGER

○ Ejercicio 1

- Para poder abrir una actividad desde una tarea, podemos añadir el siguiente código dentro del método doWork:

```
new Handler(Looper.getMainLooper()).post(() -> {  
    Intent intent = new Intent(context, SegundaActividad.class);  
    intent.setFlags(FLAG_ACTIVITY_NEW_TASK);  
    context.startActivity(intent);  
});
```



DOWNLOADMANAGER

- Gestor para descargas HTTP prolongadas
- Se encarga de:
 - realizar la descarga en segundo plano teniendo en cuenta las interacciones HTTP
 - reintentar las descargas después de fallos o durante cambios de conexión y reinicios del sistema



DOWNLOADMANAGER

- Crear objeto Request con la descarga y encolar a DownloadManager
- Cada descarga tiene una notificación asociada

```
DownloadManager.Request descarga =new DownloadManager.Request(Uri.parse("http://..."))
    .setTitle("Descarga de archivo")// Título de la notificación
    .setDescription("Descarga de archivo en curso")// Descripción de la notificación
    .setNotificationVisibility(DownloadManager.Request.VISIBILITY_VISIBLE)// Visibilidad
de la notificación
    .setDestinationUri(uri)// Uri para descargar archivo
    .setAllowedOverMetered(true);// Permitir descarga con datos móviles

DownloadManager gestorDescarga = (DownloadManager)getSystemService(DOWNLOAD_SERVICE);
descargaId = gestorDescarga.enqueue(descarga);
```



DOWNLOADMANAGER

- Se puede establecer un BroadcastReceiver para controlar el fin de la descarga

```
BroadcastReceiver descargaReceiver = new BroadcastReceiver() {
    @Override
    public void onReceive(Context context, Intent intent) {
        long id = intent.getLongExtra(DownloadManager.EXTRA_DOWNLOAD_ID, -1);

        if (descargaId == id) {
            Toast.makeText(MainActivity.this, "¡Descarga completada!",
                Toast.LENGTH_SHORT).show();
        }
    }
};

-----

//En el onCreate de la Actividad:
registerReceiver(descargaReceiver, new IntentFilter(DownloadManager.ACTION_DOWNLOAD_COMPLETE));

//En el onDestroy de la Actividad:
unregisterReceiver(descargarReceiver)
```



ALARMAS

○ Alarmas

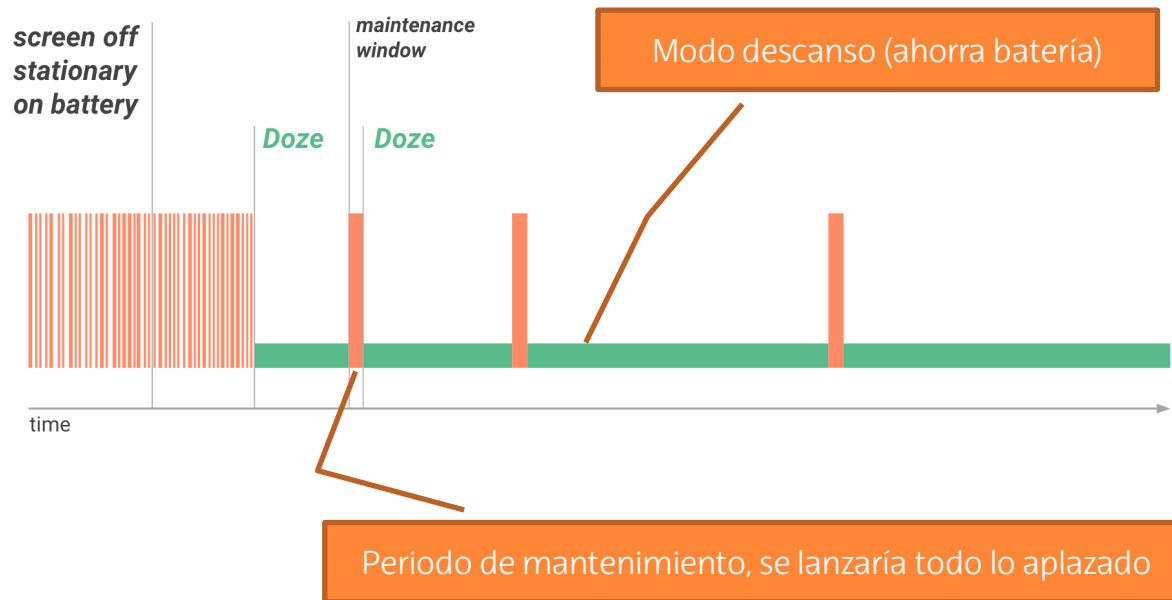
- Las alarmas permiten programar ejecuciones periódicas aunque la aplicación esté cerrada
 - En base a una hora concreta
 - En base a una repetición
- Pueden lanzar, a través de un ***PendingIntent***:
 - Un aviso Broadcast
 - Una actividad
- Las alarmas programadas se borran tras apagar el dispositivo y habría que programarlas de nuevo
 - Usando un receiver que reaccionará al evento de encender el dispositivo



ALARMAS

○ Alarmas

- Las alarmas en modo descanso (Doze) se aplazan hasta que se activa el dispositivo



ALARMAS

○ Alarmas

- Para programar una alarma se usa el ***AlarmManager*** del sistema
- Las alarmas pueden tener distintos comportamientos
 - ELAPSED_REALTIME: no despierta el dispositivo y lanza la alarma en base al tiempo que ha transcurrido desde que el dispositivo se arrancó
 - ELAPSED_REALTIME_WAKEUP: similar al anterior, pero despierta el dispositivo
 - RTC: lanza la alarma a la hora especificada, pero no despierta el dispositivo
 - RTC_WAKEUP: lanza la alarma a la hora especificada despertando el dispositivo



ALARMAS

○ Alarmas

```
Calendar calendario= Calendar.getInstance();  
calendario.set(Calendar.HOUR_OF_DAY,12);  
calendario.set(Calendar.MINUTE,28);  
calendario.set(Calendar.SECOND, 13);
```

Establecer el momento concreto en el que se quiere lanzar la alarma

Definir el PendingIntent que ejecutará la alarma

```
Intent i= new Intent(this,actividad2.class);  
PendingIntent i2= PendingIntent.getActivity(this,0,i,PendingIntent.FLAG_UPDATE_CURRENT);
```

```
AlarmManager gestor= (AlarmManager) getSystemService(Context.ALARM_SERVICE);
```

```
gestor.set(AlarmManager.RTC_WAKEUP,calendario.getTimeInMillis(),i2);
```

Obtener el AlarmManager del sistema

Establecer la alarma a una hora determinada y que despierte el dispositivo



ALARMAS

○ Alarmas

- Para que las alarmas funcionen en modo descanso, en vez de `set(..)` se usa el método `setAndAllowWhileIdle(..)`
 - Aunque puede no funcionar en todos los dispositivos
- La ejecución de las alarmas en el momento indicado es aproximada
 - Es el sistema operativo el que decide cuándo ejecutarla realmente
- Si se necesita exactitud:
 - `setExact(..)`
 - `setExactAndAllowWhileIdle(..)`



ALARMAS

○ Alarmas

- Para lograr mayor precisión y forzar que el móvil salga de modo descanso se puede utilizar **setAlarmClock()**
- Usar este método puede interferir con las políticas de ahorro de batería del teléfono

```
AlarmManager.AlarmClockInfo ac = new AlarmManager  
    .AlarmClockInfo(calendario.getTimeInMillis(), i2);  
gestor.setAlarmClock(ac, i2);
```



ALARMAS

○ Alarmas

- Para establecer que una alarma se repita
 - Intervalo mínimo 1 minuto
 - *setInexactRepeating(..)*: repite de manera aproximada
 - *setRepeating(..)*: repite de manera exacta

```
gestor.setRepeating(AlarmManager.RTC_WAKEUP, SystemClock.elapsedRealtime(), 60*1000,i2);
```

Cuándo comenzar

Repetir cada 60 segundos

```
Intent intentBC = new Intent(this, ElReceiver.class);  
intentBC.setAction("avisoPersonalizado");  
PendingIntent ibc= PendingIntent.getBroadcast(this,1,intentBC,PendingIntent.FLAG_UPDATE_CURRENT);  
gestor.setInexactRepeating(AlarmManager.RTC_WAKEUP, SystemClock.elapsedRealtime(), 3600*1000,ibc);
```

PendingIntent que lanza un Broadcast

Repetir cada hora aproximadamente



ALARMAS

○ Alarmas

- Para cancelar una alarma existente
 - Se usa el PendingIntent asociado a la alarma (o se puede crear un nuevo PendingIntent con el mismo código y el flag FLAG_NO_CREATE)

Importante, el código

```
gestor.cancel(i2);
```



ALARMAS

○ Ejercicio 2

- Crear una aplicación que establezca una alarma de modo que cada minuto, se abra la aplicación (llamando a una de sus actividades)
- Conseguir que la alarma se mantenga al reiniciar el dispositivo.
 - En este caso, como la app estará cerrada después del reinicio, la alarma puede mandar un mensaje broadcast para abrir una notificación que permita acceder a la actividad.



QUÉ USAR PARA LAS TAREAS EN BACKGROUND

Largas descargas HTTP

- DownloadManager

Trabajo diferible

- WorkManager ->PeriodicWorkRequest

Trabajo inmediato

- WorkManager->OneTimeWorkRequest

Trabajo en un instante preciso

- AlarmManager

Algunos servicios en primer plano

- WorkManager -> setForeground
- Service -> (Media playback, Activity tracking, Location Sharing, Video or Voice Calls)

